자연어처리- 토큰화

2026. 11. 9. 15:10 · ML,DL,AI 수정 삭제 공개

목차

1. 자연어처리와 토큰화
2. 단어 및 글자 토큰화
    2.1 단어 토큰화
    2.2 글자 토큰화 Character Tokenization
3. 형태소 토큰화 Morpheme Tokenization
    3.1 개념
    3.2 형태소 어휘 사전 Morpheme Vocabulary
4. 하위 단어 토큰화
    4.1 등장 배경 및 개념
    4.2 바이트 페어 인코딩 Byte Pair Encoding, BPE
    4.3 워드 피스 WordPiece

1. 자연어처리와 토큰화

자연어를 처리하는 모델을 개발하기 위해서는 3가지 문제가 해결되어야한다.

- 모호성: 인간의 언어는 맥락에 따라 여러 의미를 가진다.
- 가변성: 인간의 언어는 사투리, 강세, 신조어, 작문 스타일 로 인해 매우 가변적이다.
- 구조: 인간의 언어는 구문을 파악하여 의미를 해석한다. 문장의 구조와 문법적 요소를 이해하여 의미를 추론/분석해야한다.

이 문제를 해결하기 위해 -> 먼저 토큰으로 나눠야한다.

토큰으로 나누는 목적은 컴퓨터가 자연어를 이해할 수 있게 나누는 과정이다. 이 과정을 토큰화 라고 한다. 토큰화를 위해 토크나이저 Tokenizer 을 사용한다. 텍스트 문자열을 토큰으로 나누는 알고리즘, 소프트웨어를 의미한다.

토크나이저를 구축하는 방법은 4가지가 있다.

1. 공백 분할: 텍스트를 공백 단위로 분리한다.
2. 정규표현식 적용: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할한다.
3. 어휘 사전 적용: 사전에 정의된 단어 집합을 토큰으로 사용한다.
4. 머신러닝 활용: 토큰화 하는 방법을 학습한 머신러닝을 사용한다.

이 방법 중 어휘사전은 다음 2가지 문제를 고려해야한다.

- OOV 문제: 직접 어휘사전을 구축하기 대문에 없는 단어나 토큰이 존재할 수 있다. -> 이러한 토큰을 OOV(Out Of Vocab) 이라고 한다.
- 차원의 저주 문제(Curse Of Dimensionality): 큰 어휘사전을 구축하면(모든 토큰이나 단어를 벡터화하면) 어휘사전에 등장하는 토느개수만큼 차원이 필요하고, 벡터값이 거의 모두 0을 가지는 희소 데이터(sparse)로 표현된다.

---

2. 단어 및 글자 토큰화

2.1 단어 토큰화

텍스트 데이터를 의미있는 단위인 단어로 분리하는 작업이다.

띄어쓰기, 문장부호, 대소문자 등의 특정 구분자를 활용해 토큰화가 수행된다.

품사태깅, 개체명 인식, 기계 번역 등에서 널리 사용된다.

```
# 단어 토큰화
review= "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고"
tokenized= review.split()
print(tokenized)
# 출력: ['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어',
```

split 메서드를 통해 쉽게 토큰화할 수 있다. split 메서드에 구분자를 입력하지 않으면 공백을 기준으로 나눈...



출력 결과를 분석해보자.

- 최고! 와 최고!! 가 느낌표 하나 차이로 다른 의미 단위로 나뉜다.
- 두 단어는 비슷한 의미이지만 단어토큰화를 통해 만들어진 단어사전에는 다른 토큰이 된다.
- -> 단어토큰화는 한국어 접사, 오타, 문장부호, 띄어쓰기 오류 등에 취약하다

2.2 글자 토큰화 Character Tokenization

띄어쓰기+글자 단위로 문장을 나누는 방식이다. 비교적 작은 단어 사전을 구축할 수 있다는 장점이 있다.

언어모델링과 같은 시퀀스 예측 작업에서 활용된다.

```
# 글자 토큰화
review= "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고"
tokenized= list(review)
print(tokenized)
# 출력 ['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g
```

단어 토큰화와 달리, 공백도 토큰으로 나뉜다.

영어의 경우 -> 각 알팔벳으로 나뉜다.

한글 -> 하나의 글자는 여러 자음과 모음의 조합으로 이루어진다. -> 자소 단위로 나눠서 자소단위 토큰화를 수행한다.

```
# 자소 단위 토큰화
from jamo import h2j, j2hcj
review= "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고"
decomposed= j2hcj(h2j(review))
tokenized=list(decomposed)
print(tokenized)
# 출력: ['ㅎ', 'ㅕ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅘ', ' ', 'ㄱ', 'ㅜ', 'ㅂ', 'ㅜ',
```

비교적 적은 크기의 단어 사전을 구축할 수 있다.

그러나, 개별 토큰은 아무런 의미가 없으므로 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야한다.

토큰 조합 방식을 사용해 문장 생성, 개체명 인식 등을 구현할 수 있다.

그러나, 다의어나 동음이의어가 많은 도메인에서 약하고, 입력 시퀀스가 길어질수록 연산량이 증가한다는 단점이 있다.

3. 형태소 토큰화 Morpheme Tokenization

3.1 개념

텍스트를 형태소 단위로 나누는 방법이다. 언어의 문법과 구조를 고려해 단어를 분리하고 의미있는 단위로 분류한다.

한국어는 대부분의 언어와 달리 각 단어가 띄어쓰기로 구분되지 않는다. 어근에 다양한 접사와 조사가 조합되어 하나의 낱말을 이룬다. 따라서 각 형태소를 적절히 구분해 처리해야한다.

실제로 의미를 가지고 있는 최소 단위를 "형태소" 라고 한다. 형태소는 다음 2가지로 나뉜다.

- 자립 형태소: 스스로 의미를 가짐 free morpheme
  - 단어의 기본이 되는 형태소
  - 명사, 동사, 형용사와 같은 단어를 이룬다.
- 의존 형태소: 스스로 의미를 가지지 못하고, 다른 형태소와 조합되어 사용된다. bound morpheme
  - 자립형태소와 함께 조합되어 문장에서 특정한 역할을 한다.
  - 조사, 어미, 접두사, 접미사 등이 해당된다.

3.2 형태소 어휘 사전 Morpheme Vocabulary

자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전이다.

ex) 그는 나에게 인사를 했다. / 나는 그에게 인사를 했다.

-> 띄어쓰기를 이용한 토큰화

- 어휘사전 : [그는, 나에게, 인사를, 했다, 나는, 그에게]
- 만약, "그녀는 그에게 인사를 했다" 라는 문장이 추가되면 모델을 "그는", "나는", "그녀는" 같은 데이터를 같은 의미 단위로 인식하지 못하고 학습을 진행한다.

-> 형태소를 이용한 토큰화

- 어휘사전: [-는, -에게, 그, 나]
- 만약 "그녀는 그에게 인사를 했다" 라는 문장이 추가되면 "그녀" 의 토큰정보만 새로 학습하면 된다.

품사 태깅 POS Tagging

- 텍스트 데이터를 형태소 단위로 분석하여 각 형태소에 해당하는 품사를 태깅하는 작업이다.
- 앞 예시에 품사태깅을 수행하면 아래와 같이 태깅된다.
- 그(명사) + 는(조사) + 나(명사) + 에게(조사) + 인사(명사) + 를(조사) + 했다(동사)

---

4. 하위 단어 토큰화

4.1 등장 배경 및 개념

언어는 시간이 지남에 따라 변화하고 새로운 표현이 등장하며 더이상 사용되지 않는 표현도 생긴다.
형태소 분석기는 외래어, 띄어쓰기 오류, 오탈자가 있는 문장, 신조어가 있는 문장을 제대로 처리하지 못한다.
-> 모르는 단어를 적절한 단위로 나누는 것에 취약하며 -> 잠재적으로 어휘사전의 크기를 크게 만들고 OOV에 대응하기 어렵게 만든다.

ex) 원문: 진짜 멋있네요! 돈쭐날만 하네요 -> 결과 : [진짜, 멋있네요, !, 돈쭐날, 만, 하네요, .]

- 돈줄내다 = 돈 + 혼쭐내다 = 사회적으로 옳은 행동을 하여 귀감이 된 가게의 물건을 팔아주는 신조어
- 기존 형태소 분석기로 토큰화를 하면 **돈쭐날 + 만** 으로 분해된다.
- 돈쭐날만 이라는 문장을 토큰화한다면 **돈쭐+날+만** or **돈쭐+날만** 으로 토큰화하는 것이 적절하다
- 지금처럼 토큰화하면, 돈쭐- 이라는 어근에 올 수 있는 모든 어미에 대한 조합이 하나의 토큰으로 인식될 것이다 ex) 돈쭐내러, 돈쭐나, 돈쭐내다 등이 모두 각각 저장되는 것이다. -> 어휘사전의 크기를 크게 만든다.

하위 단어 토큰화(subword tokenization)을 통해 신조어, 오탈자, 축약어 등을 처리한다.

- 하위단어의 조합으로 나누어 토큰화한다.
- ex) reinforcement = rein + force + ment
- 단어의 길이를 줄일 수 있어서 처리속도가 빨라진다.
- 신조어, 은어, 고유어 등의 문제를 완화할 수 있다.
- **바이트 페어 인코딩, 워드 피스,** 유니그램 모델 등이 있다.

4.2 바이트 페어 인코딩 Byte Pair Encoding, BPE



없는 된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복하며 이 고자어에서 자주 등장하는 단어는 하나의 토큰으로 토큰화하고, 덜 등장하는 단어는 여러 토큰의 조합으로 표현된다.

- ex) 원문: abracadabra
- step1(ab->A): AracadAra : ab 글자쌍이 가장 빈도수가 높으므로 A로 치환한다.
- step2(ra->B): ABcadAB : ra 글자쌍이 가장 빈도수가 높으므로 B로 치환한다.
- step3(AB -> C) : CcadC : AB 글자쌍이 가장 빈도수가 높으므로 C로 치환한다.
- 더 이상 치환할 수 있는 글자 쌍이 없어서 STOP

BPE 실습

sentencepiece 라이브러리를 이용한다.
띄어쓰기나 공백도 특수문자로 취급해 토큰화 과정에서 _ 로 공백을 표현한다. 그리고 문장 맨 앞에도 _ 를 붙인다.
"Hello World" -> "Hello_World" -> "_Hello + _Wor + ld" 로 토큰화한다.

```
# 토크나이저 모델 학습
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train(
    input="../datasets/corpus.txt",
    model_prefix="petition_bpe",
    vocab_size=8000,
    model_type="bpe"
)
```

```
from sentencepiece import SentencePieceProcessor

tokenizer= SentencePieceProcessor()
tokenizer.load("petition_bpe.model") # 학습된 모델 불러오기

sentence="안녕하세요, 토크나이저가 잘 학습되었군요"
senteces=["이렇게 입력값을 리스트로 받아서,","쉽게 토크나이저를 사용할 수 있다."]

tokenzied_sentence= tokenizer.encode_as_pieces(sentence)
tokenzied_sentence= tokenizer.encode_as_pieces(senteces)
```

```
print("단일 문장 토큰화 :" ,tokenzied_sentence)
print("복수 문장 토큰화 :" ,tokenzied_sentence)
#단일 문장 토큰화 : [['_이렇게', '_입', '력', '값을', '_리', '스트', '로', '_받아서',
#복수 문장 토큰화 : [['_이렇게', '_입', '력', '값을', '_리', '스트', '로', '_받아서',
encoded_sentence= tokenizer.encode_as_ids(sentence)
encoded_sentence= tokenizer.encode_as_ids(senteces)

print("단일 문장 인코딩 :" ,encoded_sentence)
print("복수 문장 인코딩 :" ,encoded_sentence)
#단일 문장 인코딩 : [[372, 182, 6677, 4433, 1772, 1613, 6527, 4162, 6553], [1681,
#복수 문장 인코딩 : [[372, 182, 6677, 4433, 1772, 1613, 6527, 4162, 6553], [1681,
decode_ids=tokenizer.decode_ids(encoded_sentence)
decode_pieces=tokenizer.decode_pieces(tokenzied_sentence)

print("정수인코딩에서 문장 변환 :" ,decode_ids)
print("하위단어 토큰에서 문장 변환 :" ,decode_pieces)
#정수인코딩에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서,', '쉽게 토크나이저를 사용할
#하위단어 토큰에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서,', '쉽게 토크나이저를 사용
```

4.3 워드 피스 WordPiece

BPE와 유사한 방식으로 학습되지만, 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합한다.

학습 과정에서 확률적인 정보를 사용한다. 모델이 새로운 하위 단어를 생성할 때 이전 하위단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택한다.

각 글자 쌍에 대한 점수는 다음 수식으로 계산된다. f는 빈도를 나타내는 함수, x와 y는 병합하려는 하위 단어이다. f(x,y)는 x와 y가 조합된 글자 쌍의 빈도를 의미한다.

$$score = \frac{f(x,y)}{f(x),f(y)}$$

WordPiece 실습

토크나이저스tokenizers 라이브러리를 사용한다. 이 라이브러리는 정규화, 사전 토큰화를 제공한다

- 정규화: 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리 등
- 사전 토큰화: 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능이다. 공백/ 구두점을 기준으로 입력문장을 나눈다.

워드피스 토크나이저 학습

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import NFD, Lowercase, Sequence
from tokenizers.pre_tokenizers import Whitespace

tokenizer= Tokenizer(WordPiece())
tokenizer.normalizer=Sequence([NFD(), Lowercase()])
tokenizer.pre_tokenizer=Whitespace()

tokenizer.train(["../datasets/corpus.txt"])
tokenizer.save("../models/petition_wordpiece.json")
```

```
#워드 피스 토큰화
from tokenziers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder

tokenizer=Tokenizer.from_file("../models/petition_wordpiece.json")
tokenizer.decoder=WordPieceDecoder()

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있답니다"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식 :", type(encoded_sentence))

print("단일 문장 토큰화 :", encoded_sentence.tokens)
print("여러 문장 토큰화 :", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩 :", encoded_sentence.ids)
print("여러 문장 정수 인코딩 :", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환 :", tokenizer.decode(encoded_sentence.ids))
```

공감

'ML,DL,AI' 카테고리의 다른 글



자연어처리- 임베딩 →

T, Swin Transformer, CvT →

딥러닝 파이토치 교과서 6장 합성곱신경망4 →

딥러닝 파이토치 교과서 6장 합성곱신경망3 →

AI 토큰화



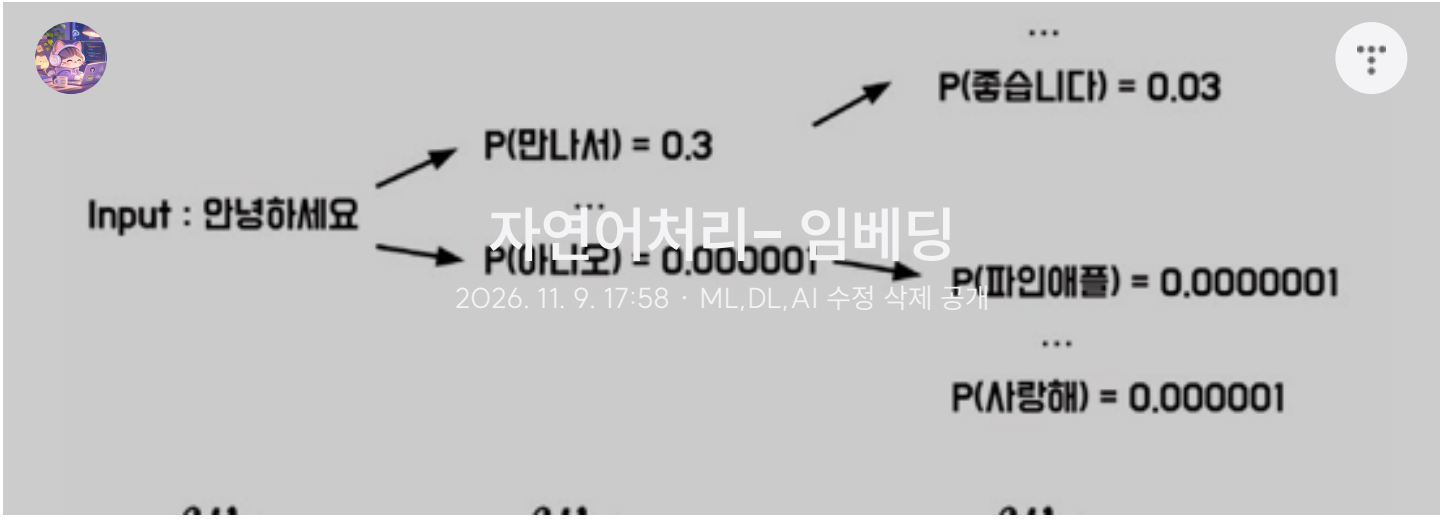
JadeO728

컴퓨터공학과 개발자 지망생



JadeO728

내용을 입력하세요.



자연어처리- 임베딩

2026. 11. 9. 17:58 · ML,DL,AI 수정 삭제 공개

목차

1. 임베딩 개념
2. 언어 모델 Language Model
2.1 자기 회귀 언어 모델
2.2 통계적 언어 모델

1. 임베딩 개념

컴퓨터는 텍스트 자체를 이해할 수 없기 때문에 텍스트 -> 숫자로 변환하는 텍스트 벡터화 과정이 필요하다

기초적인 텍스트 벡터화

1. 원-핫 인코딩 ont-hot encoding
2. 빈도 벡터화 count vectorization : 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현, 어떤 단어가 4번 등장 -> 벡터값 = 4

두 방식 다 여러 단점이 있다.

- 벡터의 희소성이 크다.
- 텍스트의 벡터가 입력텍스트의 의미를 내포하지 않는다.
- i scream for ice cream / ice cream for i scream -> 원핫과 빈도 벡터화 둘다 [1,1,1,1,1] 벡터로 표현된다.

이러한 문제를 해결하기 위해 워드 임베딩 기법을 사용한다.

- 단어의 고정된 길이의 실수 벡터로 표현

- 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현하여 단어 간의 관계를 추론한다.
- 그러나, 워드 임베딩은 고정된 임베딩을 학습하기 때문에 다의어/문맥정보를 다루기 어렵다

이 문제를 해결하기 위해 인공신경망을 사용해 동적 임베딩 기법을 사용한다.

---

2. 언어 모델 Language Model

언어모델 = 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델이다.
주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행한다.

"안녕하세요" 라는 무장 뒤에 어떤 문장이 어떤 확률로 등장할지 계산하는 것은 비지도 학습 문제이며, 계산이 사실상 불가능하다 나올 수 있는 문장이 너무 다양하기 때문이다.
-> 이러한 문제를 해결하기 위해 문장 전체를 예측하는 방법 대신, 하나의 토큰단위로 예측하는 방법인 자기회귀 언어 모델이 등장했다.

2.1 자기 회귀 언어 모델

입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측한다.
이전에 등장한 모든 토큰의 정보를 고려하여, 문장의 문맥정보를 파악하여, 다음 단어를 생성한다.
다음 단어는 다시 이전 단어를 기반으로 예측이 이루어지며, 이 과정이 반복된다.

조건부 확률 계산은 이전에 등장한 단어 시퀀스(~w(t-1))를 기반으로 다음단어 w(t)의 확률을 계산한다.
수식6.1에 조건부 확률의 연쇄법칙(chain rule for conditional probability)를 적용하면 수식6.2가 된다.

수식 6.1 언어 모델의 조건부 확률

$$P(w_t \mid w_1, w_2, \cdots, w_{t-1}) = \frac{P(w_1, w_2, \cdots, w_t)}{P(w_1, w_2, \cdots, w_{t-1})}$$

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_t \mid w_1, w_2, \cdots, w_{t-1}) = P(w_1)P(w_2 \mid w_1)\cdots P(w_t \mid w_1, w_2, \cdots, w_{t-1})$$

"안녕하세요" 다음에 등장할 수 있는 문장을 수식6.2를 기반으로 시각화하면 다음과 같다.

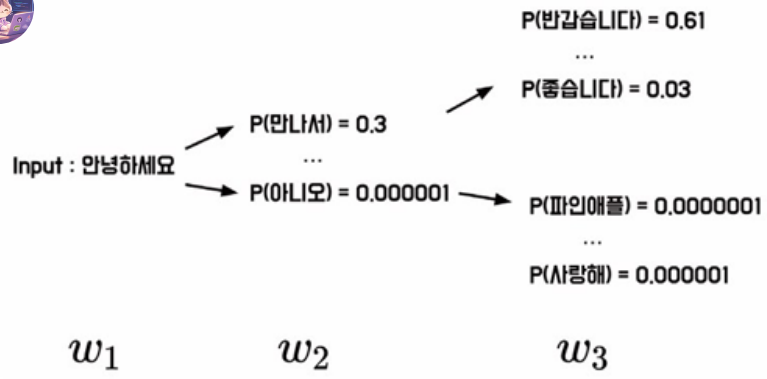


그림 6.2 조건부 확률의 연쇄법칙을 통한 자기회귀 언어 모델 구조

각 시점에서 다음에 올 토큰을 예측하는 것이 중요하다.

2.2 통계적 언어 모델

통계적 언어 모델은 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석한다.

대표적으로 마르코프 체인(Markov Chain)을 이용해 구현한다.

- 빈도 기반의 조건부 확률 모델 중 하나
- 이전상태와 현재 상태간의 전이 확률을 이용해 다음 상태를 예측한다.

빈도 기반의 조건부 확률모델이란

- 주어진 데이터에서 각 변수가 발생한 빈도수를 기반으로 확률을 계산한다.
- 전통적인 통계적 언어모델
- "안녕하세요" : 1000번 등장 // "안녕하세요 만나서" : 700번 등장 // "안녕하세요 반갑습니다" : 100번 등장
- -> 빈도 기반 조건부 확률 수식은 다음과 같다.

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서}|\text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다}|\text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- 단점1: 단어의 순서와 빈도에만 기초해 확률을 예측 -> 문맥을 제대로 파악하지 못할 수 있다.
- 단점2: 한번도 등장한 적 없는 단어, 문장에 대해서는 정확한 확률을 예측하기 어렵다 -> "데이터 희소성 문제"

- 기존에 학습한 데이터에서 패턴을 찾아 확률 분포를 생성하고
- 이를 이용하여 새로운 문장을 생성, 다양한 종류의 텍스트 데이터를 학습할 수 있다.
- 대규모 자연어 데이터를 처리하는데 효과적이다.
- 대표적 모델: GPT(트랜스포머 모델 기반 언어모델), BERT

2.2.1 N-gram

가장 기초적인 통계적 언어모델이다.

텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정한다.

연속된 N개의 단어를 하나의 단위로 취급한다.

N=1 : 유니그램, N=2 : 바이그램, N=3 : 트라이그램, N>4 : N-gram



그림 6.3 N이 1, 2, 3인 N-gram

N-gram 언어모델은 모든 토큰을 사용하지 않고 N-1개의 토큰만을 고려하여 확률을 계산한다. 따라서 수식은 다음과 같다.

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t | w_{t-1}, w_{t-2}, \cdots, w_{t-N+1})$$

N-gram의 장점

- 상대적으로 구현이 쉽다
- 작은 규모의 데이터세트에서 연속된 문자열 패턴을 분석하는데 큰 효과를 가진다.
- N-gram을 이용하면 자주 등장하는 단어의 조합을 찾을 수 있다.
- 이를 통해 '입이 무겁다'처럼 단어 하나만 봐서는 알기 어려운 관용적 표현도 발견할 수 있다.
- 단어의 순서가 중요한 자연어 처리 작업이나 문자열 패턴 분석에 활용된다.

2.2.2 TF-IDF

TF-IDF = Term Frequency-Inverse Document Frequency

텍스트 문서에서 특정 단어의 중요도를 계산하는 방법이다. 문서 내에서 단어의 중요도를 평가하는데 사용되는 통계적인 가중치를 의미한다. 즉 BoW(Bag-of-Words)에 가중치를 부여하는 방법이다.

BoW 란?

- 문서나 문장에 등장하는 단어의 빈도를 벡터값으로 기록한다.
- 따라서 BoW 벡터를 통해 영화 리뷰의 긍/부정 분류 모델을 만든다면 높은 성능이 나오기 어렵다.
- ex. i like this movie와 i don't like this movie는 don't 라는 단어가 구분하는데 결정적인 역할을 한다.
- 그러나 don't가 자주 등장하지 않아서 해당 데이터가 무시될 수 있다.
- +) BoW= 등장 빈도를 벡터값으로 표현, TF= 등장 빈도를 횟수로 표현

예를 들어, ['That movie is famous movie', 'I like that actor', 'I don't like that actor'] 말뭉치를 BoW로 벡터화하면 표 6.2와 같다.

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

2.2.2.1 단어 빈도 TF

단어 빈도(Term Frequency)는 문서 내에서 특정 단어의 빈도수를 나타낸다.

3개의 문서에서 movie 라는 단어가 4번 등장하면 movie의 TF=4 이다.

수식 6.5 단어 빈도

$$TF(t,d) = count(t,d)$$

TF가 높을 수록 특정 문서에서 중요한 역할을 할 수도 있지만,

단어 자체가 특정 문서 내에서 자주 사용되는 단어이므로 전문 용어나 관용어일 수도 있다.

단순히 등장 빈도수만 나타내어 문서의 길이가 길어질수록 해당 단어의 TF값도 높아질 수 있다.

2.2.2.2 문서빈도 DF

문서 빈도(Document Frequency, DF)는 한 단어가 얼마나 많은 문서에 나타나는지를 의미한다.

3개의 문서에서 movie 라는 단어가 4번 등장하면 movie의 DF=3이다.

수식 6.6 문서 빈도

$$DF(t,D) = count(t \in d : d \in D)$$

DF 값이 높으면 많은 문서에서 등장 => 일반적으로 널리 사용되며 중요도가 낮을 수 있다.

DF 값이 낮으면 특정 단어가 적은 수의 문서에만 등장 => 특정한 문맥에서만 사용되는 단어, 중요도가 높을 수 있다.

2.2.2.3 역문서 빈도, IDF

역문서 빈도(Inverse Document Frequency, IDF)는 전체 문서 수를 문서 빈도(DF)로 나누고 로그를 취한 값이다.

문서 내에서 특정 단어가 얼마나 중요한지를 의미한다.

문서에서 특정 단어의 등장 횟수가 적으면 IDF 값은 커진다.

수식 6.7 역문서 빈도

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

2.2.2.4 TF-IDF

TF와 IDF를 곱한 값이다.

수식 6.8 TF-IDF

$$TF - IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

문서내에서 자주 등장 + 전체 문서에서 적게 등장 -> TF-IDF 값은 커진다.

전체 문서에서 자주 등장할 확률이 높은 관사, 관용어의 값은 낮아진다.

정리하자면,

- **TF(단어 빈도)**: 특정 문서에서 단어가 얼마나 자주 등장하는지를 나타낸다

- DF(문서 빈도): 전체 문서 중 특정 단어가 등장하는 문서의 수를 나타낸다.
- **IDF(역문서 빈도)**: 단어가 전체 문서에서 얼마나 희귀한지를 나타낸다.
- **IDF가 높다**: 적은 수의 문서에 등장하므로 문서 구분에 유용할 가능성이 높다.
- **IDF가 낮다**: 많은 문서에 등장하므로 문서 구분에 유용하지 않을 가능성이 높다.
- 그리고 이 두 개념을 결합한 것이 바로 **TF-IDF = TF × IDF**
- **=> 특정 문서에서는 자주 등장하면서 전체 문서에서는 드물게 등장하는 단어에 높은 점수를 부여하는 방식이다.**

공감

'ML,DL,AI' 카테고리의 다른 글

자연어처리- 토큰화
ViT, Swin Transformer, CvT
딥러닝 파이토치 교과서 6장 합성곱신경망4
딥러닝 파이토치 교과서 6장 합성곱신경망3

AI
임베딩

JadeO728
컴퓨터공학과 개발자 지망생

JadeO728
내용을 입력하세요.